

Evaluacion Tecnica y de Calidad (QA)

Entrega Final · Dispositivos Móviles · 2026

01-EVALUACION-TECNICA-QA

AUTORES

Andrés Botero

Tecnología en Desarrollo de Software

Juan Camilo Triana

Tecnología en Desarrollo de Software

PROYECTO

OutfitCatalog · ATELIER Fashion Catalog

ASIGNATURA

Dispositivos Móviles

FECHA

Junio 2026

VERSIÓN

1.0.0

Contenido

Evaluacion Tecnica y de Calidad (QA)

1. Proposito
2. Entorno tecnico evaluado
3. Compatibilidad y fragmentacion
4. Rendimiento y estres
5. Comportamiento de red
6. Seguridad
7. Evidencia automatizada
8. Evidencia de prueba piloto con usuarios
9. Hallazgos
10. Conclusion QA

1. Proposito

Este documento evalua la calidad tecnica de OutfitCatalog considerando rendimiento, estabilidad, compatibilidad, conectividad, seguridad y pruebas. La evaluacion se realiza sobre la version final 1.0.0, generada como APK mediante EAS Build.

2. Entorno tecnico evaluado

Elemento	Valor
Framework	React Native
Plataforma de desarrollo	Expo SDK 54
Lenguaje	TypeScript
Navegacion	React Navigation native stack
Persistencia local	expo-sqlite
Almacenamiento simple	AsyncStorage
Backend remoto	Firebase Auth y Firestore
Imagenes	expo-image y Cloudinary
Red	@react-native-community/netinfo
Build Android	EAS Build
Artefacto	APK

3. Compatibilidad y fragmentacion

Android

La aplicacion fue configurada para Android con el paquete:

TEXT

com.camilotriana07.outfitcatalog

El APK final fue generado correctamente mediante EAS Build:

TEXT

<https://expo.dev/artifacts/eas/hFhAPFShte8uvkEKgF2FTW.apk>

La construccion remota finalizo con estado FINISHED, por lo cual el proyecto compila correctamente para Android en el entorno de Expo.

iOS

El proyecto esta configurado como aplicacion Expo multiplataforma y declara soporte para iOS en app.json. Tambien incluye configuracion de Google Sign-In para iOS mediante iosUrlScheme. Sin embargo, para una validacion final en iOS se requiere generar un build con cuenta Apple Developer y probarlo mediante TestFlight o dispositivo fisico.

Pantallas y tamanos

La aplicacion usa componentes responsivos de React Native, listas virtualizadas con FlatList, areas seguras con react-native-safe-area-context y layout basado en flex. Esto permite adaptacion a diferentes tamanos de pantalla. Para tablets se recomienda validacion manual adicional de densidad visual, porque el diseno principal esta optimizado para smartphones en orientacion vertical.

4. Rendimiento y estres

Optimizaciones implementadas

- Listas principales con FlatList.
- Renderizado por lotes en catalogo, inventario, favoritos y looks.
- Cache de imagenes con expo-image y politica memory-disk.
- Sincronizacion remota del catalogo limitada a prendas publicadas.
- Consulta de inventario remoto filtrada por vendorId.
- Evitacion de consultas repetidas locales mediante cargas batch por IDs.
- Sincronizacion automatica del catalogo controlada por ventana de frescura de cache.
- Persistencia local SQLite para evitar depender completamente de red.

Carga masiva de datos

El script scripts/seed-massive.js permite poblar datos de prueba:

Tipo de dato	Cantidad por defecto
Administradores	3
Vendedores	40
Clientes	250
Prendas	100
Documentos Firestore	393

Para mantener la app fluida en celulares Android de prueba, la carga recomendada mantiene el catalogo en 100 prendas:

BASH

```
npm run seed:massive -- --service-account "ruta.json" --auth-users --vendors 40 --clients 250 --admins 3 --garments 100
```

Ese escenario genera 293 usuarios y 100 prendas, para un total de 393 documentos Firestore.

Riesgos de rendimiento

Riesgo	Impacto	Mitigación
Muchas imagenes remotas	Carga visual lenta y consumo de red	Cache con expo-image, listas virtualizadas
Consultas Firestore sin filtro	Mayor latencia y costo	Consultas por published y vendorId

Riesgo	Impacto	Mitigacion
Inventario grande	Scroll lento	FlatList y carga por lotes
Sin red inicial	Catalogo vacio si no hay cache	SQLite y banner offline

5. Comportamiento de red

La aplicacion incluye monitoreo de conectividad mediante NetInfo y un OfflineBanner que informa cuando no hay conexion:

TEXT

Sin conexion - mostrando datos guardados localmente

El catalogo implementa estrategia offline-first: si Firestore no responde, se conserva la lectura desde SQLite. Las solicitudes de compra tambien tienen respaldo local cuando Firebase no esta disponible.

Escenarios evaluados conceptualmente

Escenario	Comportamiento esperado
Red disponible	Sincroniza datos remotos y actualiza cache local
Red lenta	Mantiene UI visible y muestra datos ya cargados
Sin conexion	Muestra banner offline y usa cache local
Firebase no configurado	Usa datos locales y mensajes de error legibles
Permisos Firestore rechazados	Muestra mensaje orientado a reglas de Firebase

6. Seguridad

Autenticacion

La app usa Firebase Authentication para:

- ☐ Registro por correo y contrasena.
- ☐ Inicio de sesion por correo y contrasena.
- ☐ Inicio de sesion con Google.
- ☐ Cierre de sesion remoto.

Las contrasenas no se almacenan en la app. La autenticacion se delega a Firebase Auth.

Comunicacion segura

Firebase, Cloudinary y servicios externos se consumen mediante HTTPS. Las credenciales sensibles se manejan por variables de entorno EXPOPUBLIC* o configuracion remota EAS. En una aplicacion Expo, las variables publicas no deben tratarse como secretos absolutos; las llaves realmente privadas deben mantenerse fuera del cliente.

Datos de usuario

La aplicacion guarda perfiles en Firestore con datos como nombre, correo, rol y telefono. Para produccion se recomienda:

- ☐ Mantener reglas Firestore restrictivas.
- ☐ Permitir lectura de datos personales solo al propietario o administrador.
- ☐ Evitar datos bancarios en esta version.
- ☐ Incluir politica de privacidad.
- ☐ Revisar logs para no exponer informacion sensible.

7. Evidencia automatizada

Se ejecutaron:

BASH

```
npm run build  
npm test
```

Resultado:

Verificación	Resultado
TypeScript tsc --noEmit	Aprobado
Vitest	Aprobado
Archivos de prueba	9
Casos automatizados	17
EAS Build Android	Aprobado

8. Evidencia de prueba piloto con usuarios

Ademas de la verificacion tecnica, se realizo una prueba piloto con 7 usuarios no tecnicos usando Android. La encuesta se aplico el 4 de junio de 2026 y permitio contrastar la estabilidad percibida de la app en condiciones reales de instalacion y uso.

Indicador	Resultado
Participantes	7
Instalacion exitosa	7/7
Usuarios en celular Android propio	6/7
Usuarios en celular Android prestado	1/7
Usuarios que reportaron que todo funciono bien	6/7
Usuarios que reportaron fotos faltantes	1/7
Recomendaria la app	7/7
Promedio general de calificaciones	4.91 / 5

La incidencia tecnica mas relevante fue el reporte "Algunas fotos no aparecieron". Este hallazgo se relaciona con carga de imagenes remotas y no con cierre inesperado de la aplicacion. Como mitigacion se recomienda mantener cache de imagenes, agregar placeholders mas claros y registrar errores de carga para detectar URLs rotas o fallos temporales de red.

9. Hallazgos

Hallazgo	Estado
Build Android final generado	Cumplido
Pruebas automatizadas basicas	Cumplido
Offline banner y cache local	Cumplido
Optimizacion de listas grandes	Cumplido
Prueba piloto Android con usuarios	Cumplido
Incidencia aislada de imagenes no visibles	Mejora futura
Analitica propia en Firestore	Cumplido
Crashlytics integrado	Pendiente
Firebase Analytics nativo	Pendiente
Prueba formal en iOS	Pendiente
Publicacion en tiendas	Pendiente

10. Conclusion QA

OutfitCatalog presenta una base tecnica estable para una entrega academica final. El proyecto compila, tiene pruebas automatizadas, maneja persistencia local, conectividad y build Android. La prueba piloto refuerza esta conclusion: todos los usuarios pudieron instalar la app y la mayoría no reporto fallos. Para un lanzamiento comercial se recomienda ampliar pruebas manuales en mas dispositivos reales, integrar monitoreo de crashes, registrar errores de imagenes y medir rendimiento con herramientas de profiling.